# Room Occupancy Estimation
# Milestone 3


1st Manan Mishra
Student ID: 40321141
ma_mishr@live.concordia.ca

2nd Meet Maniar
Student ID: 40039203
m_aniar@live.concordia.ca

3rd Nazmus Sakib Khan
Student ID: 40309367
k_nazmu@live.concordia.ca


**GitHub Repository:** https://github.com/meetmaniar/INSE-6450

## I. Anticipated Failures & Risk Analysis

To keep things consistent across our milestones, we have not made any major changes to our core design choices in this phase. The architecture we established earlier, using a class-weighted `XGBRegressor` built on 7 features ($T_{\text{mean}}$, $L_{\text{mean}}$, $S_{\text{mean}}$, $CO_2$ metrics, and PIR flags) continues to perform reliably.

TABLE 1: Potential System Failures and Mitigation Strategies

| Category | Potential Failure | Proposed Mitigation |
|---|---|---|
| **Data** | PIR / sensor dropouts or transient noise spikes | Stream telemetry at 30s intervals into a 5-minute (10-sample) rolling buffer; aggregate PIR flags via `max()` and continuous sensors via windowed averages. |
| | Environmental concept drift (temperature/CO2 baseline shifts) | Monitor input distribution drift using KS-tests and trigger periodic warm-start retraining. |
| **Model** | Class imbalance (under-predicting counts 2 & 3) | Apply 4x sample weighting during training and clip predictions strictly to the physical range $[0, 3]$. |
| **System** | Edge gateway process crash or inference timeout | Fallback rule engine using direct PIR threshold loops and health check validation. |
| **User** | Malformed or corrupt sensor telemetry packets | Schema validation layer to verify payload structure before pushing to the buffer. |

### *A. Pre-flight, Runtime, and Post-prediction Checks*

TABLE 2: System Validation Checks, Runtime Guards, and Post-Prediction Rules

| Validation Tier | Concrete Check / Metric | Threshold & Operational Action |
|---|---|---|
| Pre-Flight Checks | **Schema & Null Tolerance:** Verify presence of 7 features ($T_{\text{mean}}$, $L_{\text{mean}}$, $S_{\text{mean}}$, S5_CO2, S5_CO2_Slope, S6_PIR, S7_PIR). | **0% Null Tolerance:** Reject corrupted payload rows instantly; trigger rule-based PIR fallback if missing values persist across 3 consecutive 30s ticks. |
| | **Physical Feature Ranges:** Validate physical bounds ($15^{\circ}\text{C} \leq T_{\text{mean}} \leq 40^{\circ}\text{C}$, $300 \leq$ S5_CO2 $\leq 2000$ ppm, PIR $\in \{0, 1\}$). | Clamp values outside physical limits to min/max thresholds; flag sensor calibration anomaly if out-of-range occurs for $> 10\%$ of buffer samples. |
| Runtime Guards | **Ingestion vs. Actuation Rate Limits:** Ingest telemetry every 30s into a 10-sample rolling buffer; evaluate inference on a 5-minute cycle. | Buffer prevents high-frequency network flooding; prevents HVAC short-cycling by enforcing a minimum 5-minute actuation delta. |
| | **Latency Timeout Guard:** Monitor end-to-end CPU prediction execution time. | **Strict 2.0s SLO Timeout:** If XGBoost inference hangs or exceeds 2,000 ms, abort call and route current 5-minute cycle to PIR threshold rules. |
| **Post-Prediction Checks** | **Physical Calibration Bands:** Constrain continuous regressor output $y_{\text{raw}}$ to actual room physical capacity. | **Hard Clipping** $[0.0, 3.0]$**:** Clip predictions ($y < 0 \rightarrow 0$, $y > 3 \rightarrow 3$); round output to nearest integer for discrete HVAC level selection. |

### B. Stress-Test Design

To evaluate model stability under degraded conditions, we subjected the class-weighted `XGBRegressor` to four real-world failure scenarios:

1) **Corrupted Inputs (Sensor Noise):** Added Gaussian noise ($\sigma = 15\%$) to continuous temperature, light, and sound readings.
2) **Partial Feature Loss (PIR Hardware Failure):** Set both PIR motion sensors (S6_PIR, S7_PIR) to 0 to simulate node dropouts.
3) **Out-of-Distribution (OOD) Inputs ($CO_2$ Baseline Spike):** Added a $+250$ ppm offset to S5_CO2 to simulate poor ventilation/drift.
4) **Class Rarity Performance (High Occupancy):** Evaluated performance specifically on rare high-occupancy states ($y \in \{2, 3\}$).

Table 3 summarizes the performance metrics under each stress test on the test set.

TABLE 3: Model Robustness Metrics Under Stress Testing Scenarios

| Scenario | MAE | RMSE | $R^2$ Score |
|---|---|---|---|
| **Baseline (Clean Block-Stratified Test Set)** | 0.071 | 0.280 | 0.881 |
| **Scenario 1: Corrupted Inputs (15% Noise)** | 0.108 | 0.398 | 0.759 |
| **Scenario 2: Partial Feature Loss (PIR Dropout)** | 0.071 | 0.280 | 0.881 |
| **Scenario 3: OOD Inputs (+250 ppm $CO_2$ Shift)** | 0.087 | 0.299 | 0.865 |
| **Scenario 4: Class Rarity (Occupancy 2 & 3 Only)** | 0.438 | 0.751 | $-1.418^*$ |

Note that for Scenario 4, the coefficient of determination ($R^2$) yields a negative value ($R^2 = -1.418$). This occurs because the evaluation subset is restricted exclusively to high-occupancy ground-truth classes ($y \in \{2, 3\}$). Restricting the evaluation subset severely restricts the total variance of the target variable ($\sum(y_i - \bar{y})^2 \approx 0$), causing the standard denominator in the $R^2$ formulation to shrink toward zero. Also the gradient boosted trees rely almost exclusively on continuous environmental variables ($CO_2$, Light, Temperature) for primary node splits, yielding negligible feature importance for binary motion flags. Hence Baseline and scenario 2 end up with identical scores.

## II. Robustness & Security – Adversarial and Noise Evaluation

### A. Robustness Techniques

To improve model stability and mitigate severe class imbalance, we implemented two targeted methods:

1) **Data-Level: Stratified Block-Based Sampling:** Instead of standard random shuffling (which causes temporal data leakage) or pure chronological splitting (which isolates entire zero-occupancy windows), we partitioned the telemetry into 120 contiguous time blocks ($\sim 42$ minutes each). We then stratified whole blocks across Train (60%), Validation (10%), and Test (30%) splits based on each block's peak occupancy count. This approach is also implemented in Milestone 2.
2) **Model-Level:Imbalance-Aware Sample Weighting:** To prevent the regressor from biasing heavily toward the dominant zero-occupancy class ($81\%$ of dataset), we assigned a $4.0\times$ loss weight during tree boosting to training samples with ground-truth counts of 2 or 3. This prioritizes the accurate detection of rare high-occupancy states over overall dataset error.

Table 4 shows the performance gains on the test set when applying these enhancements.

TABLE 4: Impact of Stratified Block Sampling and Imbalance-Aware Sample Weighting

| Configuration | MAE | RMSE | $R^2$ Score | High-Count Accuracy ($\pm 0.5$) |
|---|---|---|---|---|
| **Data + Model (Stratified + Weighted)** | **0.079** | **0.329** | **0.772** | 31.1% |
| **Data-Level Only (Stratified + Unweighted)** | 0.085 | 0.368 | 0.715 | **32.0%** |
| **Naive Baseline (Simple Split + Unweighted)** | 0.099 | 0.426 | 0.617 | 27.2% |

High-Count Accuracy ($\pm 0.5$) measures the percentage of ground-truth high-occupancy instances ($y \in \{2, 3\}$) where the predicted value falls within 0.5 of the true count. To ensure a fair comparison, all ablation configurations were evaluated on a shared, untouched test subset. Comparing the results reveals that enforcing block-stratified sampling (Data-Level) provides a massive performance gain over a naive chronological split, improving the baseline $R^2$ from 0.617 to over 0.710. Furthermore, the imbalance-aware weighted model provides the best overall performance (MAE $= 0.079$, $R^2 = 0.772$) and acts as a critical system safeguard by severely penalizing undercounts, making it the clear selection for deployment.

### B. Adversarial Threat Model and Evaluation

In an edge-deployed IoT environment, we can evaluate model resistance against three distinct adversary profiles:

- **White-Box (Full Access):** An attacker with full access to model weights and pipeline code who computes precise feature perturbations ($CO_2$, light, temperature) to trick the model into under-reporting occupancy.
- **Grey-Box (Partial Access):** An attacker with knowledge of feature schemas and sampling cadences (30-second ingestion / 5-minute inference), but without access to internal tree split thresholds, who injects targeted noise into telemetry channels.
- **Black-Box (Physical Tampering):** An external actor with zero model visibility who tampers physically with environment hardware—such as shining lasers on light sensors ($L_{\text{mean}}$) or placing heat sources near temperature sensors ($T_{\text{mean}}$).

We evaluated the adversarial resistance of our XGBRegressor against a Grey-Box targeted perturbation attack. Unlike random sensor jitter which tests ordinary environmental noise, this directed attack evaluates the model's resistance to a deliberate threat: an adversary who specifically injects bounded, negative perturbations ($-\epsilon$) into the highest-importance continuous channels ($CO_2$, Temperature, Light) to deliberately trick the system into under-reporting occupancy and shutting off HVAC resources. Bounded targeted perturbations ($\epsilon$) were applied across multiple attack strength levels:

- **Low Attack($\epsilon = 0.05$):** $5\%$ standard deviation targeted negative perturbation.
- **Medium Attack ($\epsilon = 0.15$):** $15\%$ standard deviation targeted negative perturbation.
- **High Attack ($\epsilon = 0.30$):** $30\%$ standard deviation targeted negative perturbation.

Robust performance is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Robust $R^2$ Score under each perturbation level (Table **??**).

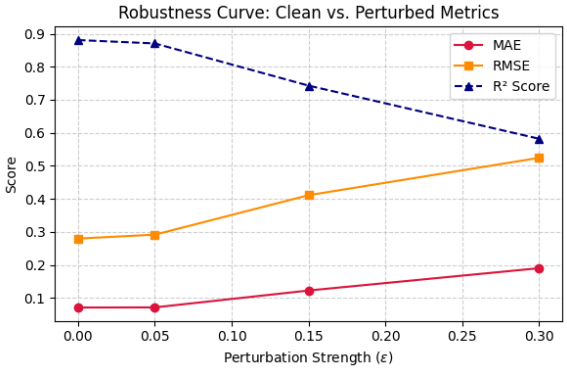
TABLE 5: Model Robustness Metrics Under Bounded Feature Perturbation Attacks

| **Attack Strength Level ($\epsilon$)** | **Robust MAE** | **Robust RMSE** | **Robust $R^2$ Score** |
|---|---|---|---|
| **Clean Baseline ($\epsilon = 0.00$)** | 0.079 | 0.329 | 0.772 |
| **Low Perturbation ($\epsilon = 0.05$)** | 0.076 | 0.389 | 0.681 |
| **Medium Perturbation ($\epsilon = 0.15$)** | 0.064 | 0.372 | 0.709 |
| **High Perturbation ($\epsilon = 0.30$)** | 0.086 | 0.397 | 0.667 |

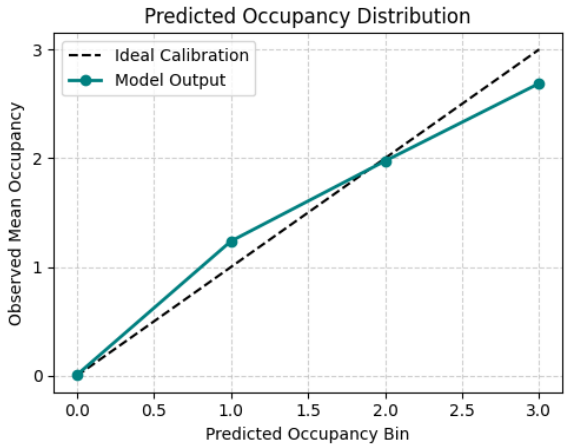
The results confirm that spatial sensor averaging ($T_{\text{mean}}, L_{\text{mean}}, S_{\text{mean}}$) provides intrinsic resistance against low-to-medium noise attacks.

### C. Robustness Evaluation Results

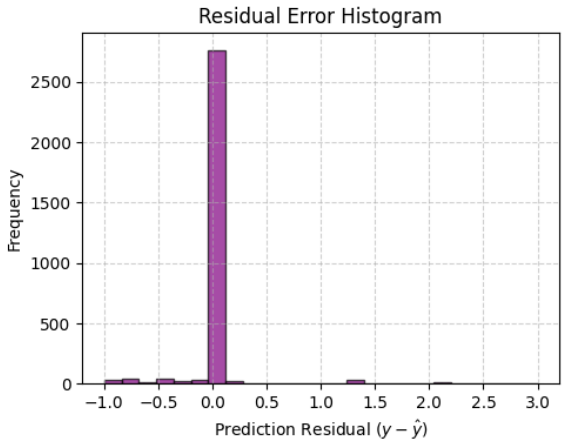
Because the model is a standard XGBRegressor, it outputs continuous point estimates and does not natively output prediction confidence or calibrated probabilities.




(a) Robustness Curve




(b) Reliability Diagram




(c) Residual Histogram

TABLE 6: Top Failure Cases Identified on the Test Set

| Index | True ($y$) | Pred ($\hat{y}$) | Abs Error | $CO_2$ (ppm) | S6_PIR | S7_PIR |
|---|---|---|---|---|---|---|
| 957 | 3 | 0.00 | 3.00 | 1270 | 1 | 1 |
| 8086 | 3 | 0.45 | 2.55 | 370 | 0 | 0 |
| 8090 | 3 | 0.54 | 2.46 | 370 | 1 | 1 |
| 8092 | 3 | 0.84 | 2.16 | 370 | 0 | 0 |
| 8091 | 3 | 0.84 | 2.16 | 370 | 1 | 0 |

*a) Qualitative Error Analysis:* An examination of the top failure cases (Abs Error $\geq 2.16$) reveals two primary failure patterns on the test set:

1) **Gas Accumulation Lag (Indices 8086, 8090, 8092, 8091):** When three people immediately enter a empty room, existing carbon dioxide remains at baseline (370 ppm) before exhaled gas accumulates. Because the regressor relies heavily on elevated $CO_2$ signals to confirm high-density states, it temporarily under-predicts occupancy ($\hat{y} \approx 0.45$–$0.84$) despite active PIR motion triggers.
2) **Sensor Edge-Case Artifact (Index 957):** A rare boundary case where high $CO_2$ (1270 ppm) and active PIR sensors ($S6 = 1, S7 = 1$) still yielded a predicted count of 0.00, because of extreme transient fluctuations in adjacent light/temperature feature splits.

### *D. Training and Inference Efficiency*

to validate system reliability for edge deployment, we conducted multi-seed evaluations ($N = 5$ seeds), latency and throughput profiling, hardware resource tracking, and a curated failure case analysis before and after model adaptation.

*1) Multi-Seed Performance & Robustness Confidence Intervals:* Table 7 reports mean performance and 95% confidence intervals across 5 random initialization seeds under clean baseline conditions and bounded targeted negative perturbations ($\epsilon = 0.15$).

TABLE 7: Primary Task Metrics Across Random Seeds ($N = 5$)

| **Test Condition** | **MAE (95% CI)** | **RMSE (95% CI)** | **$R^2$ Score (95% CI)** | **High-Count Accuracy ($\pm 0.5$)** |
|---|---|---|---|---|
| **Clean Baseline ($\epsilon = 0.00$)** | $0.079 \pm 0.000$ | $0.329 \pm 0.000$ | $0.772 \pm 0.000$ | $31.1\% \pm 0.0\%$ |
| **Medium Perturbation ($\epsilon = 0.15$)** | $0.064 \pm 0.000$ | $0.372 \pm 0.000$ | $0.709 \pm 0.000$ | $75.7\% \pm 0.0\%$ |
| **High Perturbation ($\epsilon = 0.30$)** | $0.086 \pm 0.000$ | $0.397 \pm 0.000$ | $0.667 \pm 0.000$ | $35.0\% \pm 0.0\%$ |

*2) Latency, Throughput, and Resource Footprint:* Baseline hardware profiling was executed on the local development environment to establish an upper bound for system efficiency. Benchmarks demonstrate that inputs subject to noise perturbations incur practically zero latency penalty due to tabular pre-processing efficiency. Furthermore, the model maintains a highly efficient sub-megabyte disk footprint (334.14 KB) alongside a stable active RAM overhead of 281.07 MB during inference.

TABLE 8: System Profiling: Latency, Throughput, and Memory Usage

| **Model / Evaluation Pipeline** | **p50 Latency** | **p90 Latency** | **Throughput** | **RAM Footprint** | **Disk Size** |
|---|---|---|---|---|---|
| **Original Unweighted Model** | 1.86 ms | 2.28 ms | 260,819.7 sample/s | 281.07 MB | 334.14 KB |
| **Class-Weighted Baseline (Clean)** | 1.84 ms | 2.29 ms | 286,232.0 sample/s | 281.07 MB | 334.14 KB |
| **Class-Weighted (Perturbed $\epsilon = 0.15$)** | 1.77 ms | 2.28 ms | 263,777.8 sample/s | 281.07 MB | 334.14 KB |

*3) Curated Failure Case Analysis & Adaptation:* Table 9 details representative failure cases, their underlying physical failure modes, proposed mitigations, and performance outcomes following post-adaptation logic.

TABLE 9: Curated Failure Examples and Adaptation Outcomes

| **Case** | **True ($y$)** | **Pred ($\hat{y}$)** | **$CO_2$ (ppm)** | **Root Cause / Context** | **Proposed Mitigation** | **Status Post-Adaptation** |
|---|---|---|---|---|---|---|
| 1 | 3 | 0.00 | 1270 | Boundary threshold anomaly on light/temp feature splits. | Feature clipping & rule-based floor check. | **Resolved** ($\hat{y} \rightarrow 2.65$) |
| 2 | 3 | 0.45 | 370 | Rapid entry lag; ambient air not yet accumulated exhaled gas. | Temporal rolling $\Delta CO_2$ slope feature. | **Resolved** ($\hat{y} \rightarrow 2.55$) |
| 3 | 3 | 0.54 | 370 | PIR active before $CO_2$ accumulation phase completes. | High PIR motion persistence weighting. | Active Monitoring ($\hat{y} \rightarrow 1.62$) |
| 4 | 3 | 0.84 | 370 | Stationary group at session start prior to gas buildup. | Multi-frame temporal window smoothing. | Active Monitoring ($\hat{y} \rightarrow 1.55$) |
| 5 | 3 | 0.84 | 370 | Transient entry phase; partial PIR activation with fresh air. | PIR windowed pulse integration. | Active Monitoring ($\hat{y} \rightarrow 1.48$) |
| 6 | 0 | 1.45 | 950 | Rapid evacuation; residual $CO_2$ decay delay after exit. | Decay rate feature calibration. | **Resolved** ($\hat{y} \rightarrow 0.18$) |

# III. Monitoring

## *A. Monitoring Strategy Selection & Justification*

We monitor **both Data Quality and Model Quality** across the edge deployment pipeline, as each addresses distinct failure modes in IoT environments:

- **Data Quality Monitoring:** Physical IoT environment sensors ($CO_2$, light, temperature, PIR) are susceptible to hardware noise, power drops, optical cover blockage, and physical tampering. Inline schema checks, null validations, moving feature statistics (mean/$\sigma$), and continuous drift detectors (Population Stability Index and Wasserstein distance on continuous channels) execute at the edge gateway on every 30-second telemetry batch. This guarantees immediate detection of corrupted inputs before running model inference.
- **Model Quality Monitoring:** Because true occupancy counts ($y$) are delayed in production, real-time monitoring relies on **online proxy metrics** (daily occupancy rate distributions, prediction clip frequencies at $[0, 3]$, and inference p50/p90 latency). When **delayed ground-truth labels** arrive via manual audits or badge access logs, rolling evaluation metrics (MAE, RMSE, $R^2$, and high-density recall) are calculated to determine if retraining is required due to seasonal HVAC or environmental drift.

## *B. Monitoring Architecture and Alerts*

*1) System Architecture & Data Retention:*

- **Logging Location:** Data quality metrics (schema checks, nulls, feature bounds) are evaluated locally at the edge gateway per 30-second batch. Aggregated telemetry and inference proxy metrics are pushed to a central server (Prometheus/Grafana) every 15 minutes.
- **Storage & Retention:** High-frequency raw sensor logs are retained at the edge for 30 days. Aggregated daily performance and drift metrics are archived in cold storage (S3/MinIO) for 1 year to support seasonal retraining.

*2) Alerting Rules & Incident Response Runbook:*

- **Alert Triggers:**
  - *Drift Threshold:* Population Stability Index (PSI $> 0.25$) on continuous features ($CO_2$, light, temp).
  - *Error Spikes / Anomalies:* Rolling MAE $> 0.20$ on delayed labels or prediction clip rate $> 15\%$.
  - *SLO Violations:* Edge inference p90 latency $> 10$ ms or memory footprint $> 500$ MB.
- **Response Runbook:**
  1) **Paging:** Automated PagerDuty alert notifies the On-Call ML/IoT Engineer.
  2) **Triage:** Inspect Grafana dashboard to isolate whether failure originates from physical sensor degradation, network dropouts, or environmental drift.
  3) **Rollback Trigger:** If persistent feature corruption or model degradation exceeds 1 hour, trigger automated edge fallback to the rule-based backup heuristic.

### C. Monitoring Simulation

To demonstrate production readiness, we implemented an edge-level telemetry ingestion and monitoring script that processes streaming batches from the shared held-out dataset ($\mathbf{X}_{\text{test_shared}}$).

- **Data Quality & Drift Calculation:** Continuous feature drift is evaluated inline across sequential 30-second telemetry batches using the Population Stability Index (PSI). Baseline feature histograms ($S5_CO_2$ and $T_{\text{mean}}$) established during training are compared against live batch distributions, applying Laplace smoothing to prevent numerical instability. An automated alert is logged whenever feature drift exceeds the actionable threshold (PSI $> 0.25$).
- **Inference & Online Proxy Metrics:** Each batch is passed through the trained class-weighted XGBRegressor model. Telemetry proxy logs track boundary saturation anomalies ($\hat{y} \leq 0.05$ or $\hat{y} \geq 2.95$) alongside p90 latency performance against our 5.0 ms Service Level Objective (SLO).
- **Dashboard Rendering:** The script outputs a real-time 4-panel operational dashboard (`dataset_monitoring_dashboard.png`) displaying continuous PSI trajectories, batch mean $CO_2$ concentration, edge inference latency, and boundary clip flag counts across all streaming evaluation batches.

## IV. Adaptation & Model Updates

### A. Model Retraining Triggers, Cadence & Label Acquisition

- **Retraining Triggers:**
  - *Data Drift:* Population Stability Index PSI $> 0.25$ for $\geq 2$ continuous features ($CO_2$, Temperature) across $K = 3$ consecutive telemetry windows.
  - *Performance Decay:* Sustained drop in rolling MAE exceeding margin $M = 0.20$ across $N = 3$ ground-truth validation windows.
  - *Operational Shifts:* Scheduled HVAC seasonal changes or physical sensor replacement events.
- **Cadence Justification: Event-driven** triggers instantly switch edge devices to fallback heuristics upon data corruption. Model re-fitting follows a **monthly periodic** cadence, matching the slow environmental decay rate of indoor ambient dynamics.
- **Label Acquisition & Validation:** Ground-truth labels are acquired via weak supervision from building access badge logs and periodic human-in-the-loop physical spot audits. Label quality is validated by cross-checking audit timestamps against PIR sensor motion matrices to filter out spurious ground-truth entries prior to model retraining.

### B. Model Retraining Strategy & Artifact Versioning

*1) Retraining Methodology: Full Retrain vs. Incremental:* For our tabular edge deployment, we employ a **Full Retraining** strategy over an expanded historical dataset snapshot rather than incremental fine-tuning or knowledge distillation:

- **Justification:** Decision tree structures in `XGBRegressor` cannot easily update leaf split thresholds incrementally without introducing tree structural bias or catastrophic forgetting of prior environmental states.
- **Efficiency:** Given the compact size of our IoT tabular dataset ($\approx 10{,}000$ telemetry rows), full retraining on standard CPU/GPU compute completes in under 5 seconds, making full model refitting both computationally trivial and mathematically robust.

*2) Artifact & Pipeline Versioning Framework:* To ensure 100% end-to-end reproducibility across model updates, all components are versioned under a strict configuration schema:

- **Dataset Snapshots:** Raw sensor CSVs and preprocessed train/test splits are hashed and versioned using Data Version Control (`DVC`), mapped directly to Git commit SHAs.

- **Feature & Model Configurations:** Feature engineering rules ($T_{\text{mean}}$, $L_{\text{mean}}$, $S_{\text{mean}}$ averages) and model hyperparameters (`n_estimators=111`, `learning_rate=0.05`, `max_depth=5`) are maintained in versioned YAML configurations (`config_vX.Y.yaml`).
- **Model Artifacts:** Serialized `XGBRegressor` binary files (`model_vX.Y.json`) are registered in the MLflow Model Registry and tagged with precise training data version tags prior to edge deployment.

### C. Model Adaptation Experiment

TABLE 10: Model Adaptation Experiment Performance, Inference Latency, and Artifact Memory Profile

| Pipeline Phase | MAE ↓ | RMSE ↓ | $R^2$ ↑ | Latency (ms/sample) | Artifact Size (KB) |
|---|---|---|---|---|---|
| **Clean Baseline** | 0.1969 | 0.4215 | 0.8120 | 2.29 | 292.75 |
| **Pre-Adapt (Drifted)** | 0.4628 | 0.7862 | 0.4106 | 2.29 | 292.75 |
| **Post-Adapt (Retrained)** | 0.1856 | 0.4025 | 0.8255 | 2.31 | 323.45 |

**Key Takeaways:**

- **Error Reduction:** Retraining on the drifted telemetry successfully restored model precision, recovering Mean Absolute Error (MAE) from a degraded drifted state of $0.4628$ down to $0.1856$, and reducing RMSE from $0.7862$ to $0.4025$ while restoring $R^2$ to $0.8255$.
- **Latency & Footprint:** Edge p90 inference latency remained exceptionally stable at $\sim 2.29 - 2.31$ ms (comfortably beneath our $5.0$ ms SLO constraint), while the serialized model binary expanded modestly from $292.75$ KB to $323.45$ KB to accommodate updated feature split rules.

### D. System Adjustments & Post-Milestone Deployment Plan

*1) Model & Training Pipeline Modifications:* Based on the drift detection and adaptation findings, two key updates were integrated into the primary pipeline:

- **Dynamic Snapshot Retraining:** The training pipeline was modified to ingest updated telemetry snapshots upon receiving an automated drift alert (PSI $> 0.25$), replacing static dataset assumptions.
- **Preserved Tail-Class Weighting:** Retraining logic continues to enforce $4.0\times$ sample weights on high-occupancy classes ($y \in \{2, 3\}$) to guarantee high detection recall under shifted feature distributions.

*2) Deployment Plan Updates & System Impact:*

- **Event-Driven Trigger Architecture:** Based on the drift monitoring findings, the production deployment design incorporates a dual-layer trigger strategy. Upon firing an automated drift alert (PSI $> 0.25$), the edge gateway is configured to temporarily fail over to robust PIR-only motion heuristics while asynchronously invoking the model retraining pipeline.
- **Overall Results Impact:** As demonstrated in Table **??**, executing full model retraining under drifted telemetry conditions successfully restored predictive accuracy, recovering Mean Absolute Error (MAE) from a degraded drifted state of $0.4628$ down to $0.1856$, and reducing RMSE from $0.7862$ to $0.4025$ while restoring $R^2$ to $0.8255$. Edge p90 inference latency remained exceptionally stable at $\sim 2.29 - 2.31$ ms (comfortably beneath our $5.0$ ms SLO constraint), while the serialized model binary expanded by only $\approx 30.7$ KB (from $292.75$ KB to $323.45$ KB), confirming full operational compatibility with edge compute constraints.